RS-Rently

Distribuirani rent-a-car sustav

zasnovan na mikroservisnoj arhitekturi

Kolegij: Raspodijeljeni sustavi 2025./2026.

Petar Ally

11. travnja 2026.

Sadržaj

1 Uvod **3**

2 Arhitektura sustava **3**
2.1 Auth-service . . . 3
2.2 Booking-service . . . 3
2.3 Damage-service . . . 4
2.4 Mail-worker . . . 4
2.5 GPS-tracker . . . 4
2.6 Frontend . . . 4
2.7 Infrastruktura . . . 5
2.8 Cross-Origin Resource Sharing . . . 5

3 Tehnologije **5**

4 Funkcionalnosti **5**
4.1 Autentifikacija i autorizacija . . . 6
4.2 Rezervacija vozila . . . 6
4.3 Evidencija šteta . . . 6
4.4 GPS praćenje . . . 6
4.5 Praćenje grešaka . . . 6

5 Komunikacijski protokoli **7**
5.1 REST API (HTTP/JSON) . . . 7
5.2 gRPC (Protocol Buffers) . . . 8
5.3 Redis Message Queue . . . 8

6 Struktura projekta **8**

7 Docker i pokretanje **9**

8 Dijagram klasa **10**

9 UML dijagram komponenti **11**

10 Sekvencijski dijagram **12**

11 Zaključak **13**

1 Uvod

Suvremene web aplikacije rijetko se grade kao monolitni sustavi. Umjesto toga, složenost se razlaže na manje, samostalne cjeline – *mikroservise* – koji komuniciraju putem jasno definiranih sučelja. Takav pristup donosi neovisnost razvoja i deploya, mogućnost skaliranja pojedinog dijela sustava te slobodu odabira tehnologije koja najbolje odgovara konkretnom zadatku.

RS-Rently nastao je upravo iz te ideje. Riječ je o distribuiranom sustavu za iznajmljivanje vozila koji objedinjuje šest mikroservisa, svaki zadužen za jednu domenu: autentifikaciju, rezervacije, evidenciju šteta, slanje obavijesti, GPS praćenje i korisničko sučelje. Servisi međusobno komuniciraju putem tri različita protokola – REST, gRPC i Redis message queue – čime projekt demonstrira raznolikost komunikacijskih obrazaca prisutnih u stvarnim distribuiranim sustavima.

Cilj projekta nije samo implementacija funkcionalne aplikacije, već i prikaz ključnih koncepata raspodijeljenih sustava: asinkrone obrade, odvajanja producenta i konzumenta poruka, kontejnerizacije, simulacije cloud infrastrukture te centraliziranog praćenja grešaka. Sustav se u cijelosti pokreće jednom `docker-compose` naredbom, a svaki servis živi u vlastitom kontejneru, s vlastitim ovisnostima i životnim ciklusom.

2 Arhitektura sustava

Arhitektura RS-Rently slijedi mikroservisni obrazac u kojem je svaka poslovna funkcija izdvojena u zaseban proces. Servisi ne dijele ni bazu podataka ni memoriju – jedina sprega među njima je mrežna komunikacija putem strogo definiranih API-ja. Svih šest servisa i dvije infrastrukturne komponente povezani su zajedničkom Docker mrežom `rent_network`, unutar koje se međusobno pronalaze putem Docker DNS rezolucije.

2.1 Auth-service

Auth-service predstavlja ulaznu točku sustava za svakog korisnika. Implementiran je u FastAPI-ju i izlaže dva endpointa: `POST /login` za prijavu te `GET /verify` za provjeru valjanosti tokena. Prijava se temelji na hardkodiranim kredencijalima (`admin/admin`) što je dovoljno za demonstraciju, a u produkcijskoj varijanti zamijenilo bi se bazom korisnika. Nakon uspješne provjere, servis generira JWT token potpisan HS256 algoritmom, s rokom trajanja od jednog sata. Taj token postaje jedini dokaz identiteta – svi ostali servisi očekuju ga u `Authorization` zaglavlju zahtjeva.

2.2 Booking-service

Booking-service prima zahtjeve za rezervaciju vozila na `POST /bookings` endpointu. Prije obrade provjerava prisutnost JWT tokena u zaglavlju te valjanost ulaznih parametara (`car_id` i `user_email`). Za svaku novu rezervaciju generira UUID identifikator, konstruira JSON objekt s podacima o rezervaciji i gura ga u Redis listu `email_queue` naredbom `RPUSH`. Korisniku odmah vraća potvrdu s identifikatorom rezervacije, dok se daljnja obrada (slanje e-maila) odvija potpuno asinkrono, bez blokiranja odgovora. Ova arhitektonska odluka ključna je za responsivnost sustava – korisnik ne čeka da se e-mail pošalje da bi dobio potvrdu.

2.3 Damage-service

Damage-service zadužen je za evidenciju oštećenja vozila putem uploada fotografija. Izlaže `POST /upload-damage` endpoint koji prima datoteku u `multipart/form-data` formatu. Primljenu datoteku pohranjuje u S3 bucket nazvan `stete-bucket`, pri čemu se koristi LocalStack kao lokalna simulacija AWS-a. Servis također sadrži zasebnu Lambda funkciju (`lambda-function.py`) koja bi na stvarnom AWS-u bila pokrenuta S3 eventom nakon svakog novog uploada – u toj funkciji moguće je implementirati automatsku klasifikaciju štete, obavijest osiguranja ili bilo kakvu drugu obradu.

2.4 Mail-worker

Mail-worker je jedini servis koji ne izlaže nikakav API. Umjesto toga, radi kao pozadinski proces u beskonačnoj petlji, neprekidno osluškujući Redis red `email_queue` pomoću blokirajućeg čitanja (`BLPOP`) s timeoutom od 5 sekundi. Kada poruka stigne, parsira JSON podatke, izvlači e-mail adresu i identifikator vozila te simulira slanje e-maila (u ovoj verziji to je `time.sleep(2)` s ispisom u log). Između svake iteracije provjerava i Redis ključ `crash_mail_worker` – mehanizam koji omogućuje da Booking-service na zahtjev izazove simulirani pad workera, što je korisno za demonstraciju Sentry integracije.

Ovaj producer-consumer obrazac osigurava otpornost sustava: čak i ako Mail-worker privremeno padne, poruke ostaju u Redis redu i bit će obrađene nakon ponovnog pokretanja. Redis pritom služi i kao prirodni buffer koji apsorbira nagle skokove u broju rezervacija.

2.5 GPS-tracker

GPS-tracker jedini je servis koji ne koristi REST, već gRPC protokol. Sučelje je definirano Protocol Buffers shemom s jednom RPC metodom `SendLocation` koja prima strukturiranu poruku `LocationUpdate` (identifikator vozila, geografska širina i dužina) i vraća praznu poruku `Empty`. Servis koristi `ThreadPoolExecutor` s 10 radnih niti, čime može paralelno obrađivati lokacijske podatke od više vozila.

Izbor gRPC-a za ovaj servis motiviran je prirodom podataka: GPS koordinate su jednostavne strukturirane poruke koje se šalju u visokoj frekvenciji, a gRPC nudi značajno manje overheada od REST-a zahvaljujući binarnoj serijalizaciji Protocol Buffersa i HTTP/2 transportu.

2.6 Frontend

Frontend je izgrađen u Vue.js 3 koristeći Composition API. Aplikacija se sastoji od pet komponenti: `Login` za prijavu, `Dashboard` za pregled sustava, `Bookings` za kreiranje rezervacija, `DamageUpload` za upload slika šteta i `CrashControls` za testiranje padova servisa. Vue Router štiti zaštićene rute provjerom prisutnosti tokena u `localStorage` – ako token ne postoji, korisnik se preusmjerava na stranicu za prijavu.

Komunikacija s backendom apstrahirana je kroz centralni modul `api.js` koji kreira tri Axios instance – po jednu za svaki REST servis. Interceptor automatski dodaje JWT token u zaglavlje svakog zahtjeva prema Booking i Damage servisima, čime se eliminira potreba za ručnim upravljanjem autentifikacijom u svakoj komponenti.

U produkciji se Vue aplikacija kompajlira u statičke datoteke i servira putem Nginx-a, koji ujedno brine o SPA routingu (sve rute preusmjerava na `index.html`), GZIP kompresiji i

cachiranju statičkih resursa.

2.7 Infrastruktura

Dva infrastrukturna servisa čine temelj na kojem ostali servisi grade svoju funkcionalnost. **Redis** (Alpine varijanta) služi kao message broker za asinkronu komunikaciju između Booking-servisa i Mail-workera. Booking-service piše poruke u Redis listu, a Mail-worker ih čita – time se ostvaruje potpuno odvajanje producenta i konzumenta. **LocalStack** simulira AWS ekosustav; u `docker-compose.yaml` aktivirani su `dynamodb` i `s3` servisi, pri čemu trenutna implementacija koristi isključivo S3 (bucket `stete-bucket`) za pohranu fotografija šteta, dok je DynamoDB pripremljen kao prostor za buduće proširenje (npr. perzistencija rezervacija). Damage-service se prema S3 API-ju odnosi identično kao prema pravom AWS-u – razlikuje se samo `endpoint_url`.

2.8 Cross-Origin Resource Sharing

Budući da se frontend (`:3000`) i backend servisi (`:8000`, `:8001`, `:8002`) u razvojnom okruženju poslužuju s različitih portova, svaki zahtjev iz preglednika je tehnički *cross-origin* prema SOP-u (Same-Origin Policy). Kako bi se izbjegle greške tipa *CORS preflight failed*, svi FastAPI servisi uključuju `CORSMiddleware` s permisivnom konfiguracijom (`allow_origins=["*"]`). Ovakva postavka prihvatljiva je u demonstracijskom okruženju; u produkciji bi se ograničila na konkretnu domenu frontenda.

3 Tehnologije

Svi backend servisi napisani su u **Python 3.10**. Za tri servisa koji izlažu REST API odabran je **FastAPI** – asinkroni web framework koji uz minimalan kod pruža automatsku validaciju podataka putem Pydantic modela, automatsko generiranje OpenAPI (Swagger) dokumentacije i visoke performanse temeljene na Starlette/Uvicorn stacku. GPS-tracker koristi **gRPC** s **Protocol Buffers** definicijama sučelja, što omogućuje strogo tipiziranu, binarno serijaliziranu komunikaciju s nižim overheadom od JSON-a.

Za asinkronu komunikaciju između servisa koristi se **Redis**, in-memory pohrana podataka koja ovdje funkcionira kao message broker. Pristup AWS resursima ostvaruje se putem **boto3** SDK-a, dok **LocalStack** omogućuje lokalno pokretanje S3 i DynamoDB servisa bez oblačne infrastrukture. Autentifikacija se temelji na **JWT** tokenima implementiranim pomoću **PyJWT** biblioteke.

Frontend koristi **Vue.js 3** s Composition API-jem, **Vue Router** za upravljanje rutama i **Axios** kao HTTP klijent. Build proces odvija se putem **Vite** alata, a produkcijsko posluživanje preuzima **Nginx**. Svi servisi pakirani su u **Docker** kontejnere i orkestrirani putem **Docker Compose-a**. Za centralizirano praćenje grešaka integriran je **Sentry** sa StarletteIntegration na backendu i browserTracingIntegration na frontendu.

4 Funkcionalnosti

4.1 Autentifikacija i autorizacija

Tok autentifikacije započinje unosom korisničkog imena i lozinke na Login stranici. Frontend šalje `POST /login` zahtjev Auth-servisu koji provjerava kredencijale i, ako su ispravni, vraća JWT token s `sub` claimom (korisničko ime) i `exp` claimom (istek za 1 sat). Frontend pohranjuje token u `localStorage` i od tog trenutka ga automatski uključuje u svaki zahtjev prema zaštićenim servisima putem Axios interceptora. Vue Router dodatno štiti zaštićene rute – navigation guard provjerava prisutnost tokena prije svakog prijelaza na zaštićenu stranicu.

4.2 Rezervacija vozila

Korisnik na stranici za rezervacije odabire jedno od pet dostupnih vozila (BMW X5, Audi A6, Mercedes C200, VW Golf, Škoda Octavia), unosi e-mail adresu, datume preuzimanja i vraćanja te opcionalne podatke poput broja putnika i napomena. Backend API trenutno prihvaća samo dva query parametra – `car_id` i `user_email` – dok dodatna polja (datumi, broj putnika, telefon, napomena) ostaju na razini prezentacijskog sloja i prikazuju se u potvrdi; ovakva jasna razdvojenost ostavlja prostor za proširenje API-ja bez promjene UI-a.

Booking-service prima zahtjev, generira UUID identifikator, konstruira JSON poruku i stavlja je u Redis red. Korisniku se odmah prikazuje potvrda s identifikatorom i statusom `confirmed`. U pozadini, Mail-worker preuzima poruku iz reda i simulira slanje potvrdnog e-maila. Ovakva asinkrona obrada osigurava da korisnik nikada ne čeka na sporedne operacije poput slanja e-maila.

4.3 Evidencija šteta

Prijava štete svodi se na odabir fotografije oštećenja u jednom od podržanih formata (JPG, PNG, GIF) i upload putem web sučelja. Frontend šalje datoteku kao `multipart/form-data` na Damage-service koji je prosljeđuje u S3 bucket. Korisnik dobiva potvrdu sa statusom `pending`, signalizirajući da je slika zaprimljena i čeka obradu. U produkcijskom okruženju, S3 event bi pokrenuo Lambda funkciju koja bi mogla provesti automatsku analizu slike.

4.4 GPS praćenje

GPS-tracker prima lokacijske podatke putem gRPC poziva. Svaki poziv sadrži identifikator vozila i GPS koordinate. Servis bilježi primljene lokacije, čime pruža temelj za praćenje flote vozila u stvarnom vremenu. Korištenje gRPC-a umjesto REST-a opravdano je prirodom podataka – GPS koordinate su male, strukturirane poruke koje se šalju u visokoj frekvenciji, a binarna serijalizacija Protocol Buffersa i multipleksiranje HTTP/2 protokola značajno smanjuju mrežni overhead.

4.5 Praćenje grešaka

Svaki servis integriran je sa Sentry platformom za praćenje grešaka. Sentry konfiguracija čita se iz varijabli okruženja: `SENTRY_DSN` (ciljni projekt), `SENTRY_ENV` (okruženje, default `development`) te `SENTRY_TRACES_SAMPLE_RATE` (udio zahtjeva koji se uzorkuje za performance tracing, default `0.0`). Inicijalizacija je omotana u `try/except` blok pa se servisi čisto pokreću i u odsustvu DSN-a – Sentry tada tiho postaje no-op.

Backend servisi koriste `StarletteIntegration` (za REST servise), `StdlibIntegration` i `LoggingIntegration`, dok frontend koristi `browserTracingIntegration` za praćenje performansi i `replayIntegration` za snimanje korisničkih sesija prilikom grešaka. Zastavica `send_default_pii=True` omogućuje da Sentry automatski prikuplja osnovne podatke o korisniku (IP, user-agent, email ako je postavljen).

Svaki servis nudi `GET /__debug/crash` endpoint za namjerno izazivanje iznimke i testiranje Sentry pipeline-a. Mail-worker, koji nema HTTP sučelje, podržava udaljeni crash signal putem Redis ključa `crash_mail_worker`; Booking-service postavlja taj ključ preko vlastitog `POST /__debug/crash-mail-worker` endpointa, a worker ga obrađuje na idućoj BLPOP iteraciji. Frontend komponenta `CrashControls` objedinjuje sve ove akcije u jednostavno sučelje s četiri gumba.

5 Komunikacijski protokoli

Jedna od ključnih odluka u dizajnu distribuiranog sustava jest odabir komunikacijskog protokola. RS-Rently namjerno koristi tri različita protokola kako bi demonstrirao da ne postoji univerzalno rješenje – svaki protokol ima prednosti koje ga čine optimalnim za određeni tip komunikacije.

5.1 REST API (HTTP/JSON)

Auth-service, Booking-service i Damage-service komuniciraju s frontendom putem REST API-ja. Zahtjevi i odgovori razmjenjuju se u JSON formatu, a FastAPI automatski generira interaktivnu Swagger dokumentaciju na `/docs` putanji svakog servisa. REST je odabran za ove servise jer je intuitivno razumljiv, izvrstan za CRUD operacije i nativno podržan u web preglednicima. Tablica 1 prikazuje pregled svih REST endpointa.

Tablica 1: Pregled REST API endpointa

Servis	**Metoda**	**Endpoint**	**Opis**
Auth	POST	/login	Prijava korisnika, vraća JWT token
Auth	GET	/verify	Provjera valjanosti JWT tokena
Auth	GET	/__debug/crash	Simulirani pad servisa (Sentry test)
Booking	POST	/bookings	Kreiranje nove rezervacije vozila
Booking	GET	/__debug/crash	Simulirani pad servisa (Sentry test)
Booking	POST	/__debug/crash-mail-worker	Postavlja Redis signal za pad Mail-workera
Damage	POST	/upload-damage	Upload slike oštećenja na S3
Damage	GET	/__debug/crash	Simulirani pad servisa (Sentry test)

5.2 gRPC (Protocol Buffers)

GPS-tracker koristi gRPC – framework za udaljene pozive procedura temeljen na HTTP/2 transportu i Protocol Buffers serijalizaciji. Sučelje servisa definirano je `.proto` datotekom iz koje se automatski generira klijentski i serverski kod:

```
syntax = "proto3";

service GPSTracker {
  rpc SendLocation (LocationUpdate) returns (Empty) {}
}

message LocationUpdate {
  string car_id = 1;
  double latitude = 2;
  double longitude = 3;
}

message Empty {}
```

Listing 1: Definicija gRPC sučelja (gps.proto)

Prednost gRPC-a pred REST-om u ovom kontekstu je trostruka: binarna serijalizacija daje manje poruke nego JSON, HTTP/2 omogućuje multipleksiranje više poziva preko jedne TCP veze, a strogo tipizirana shema eliminira cijelu kategoriju grešaka pri serijalizaciji.

5.3 Redis Message Queue

Komunikacija između Booking-servisa i Mail-workera odvija se putem Redis liste `email_queue`. Booking-service dodaje poruke naredbom `RPUSH`, a Mail-worker ih čita blokirajućom naredbom `BLPOP` s timeoutom od 5 sekundi. Ovo je klasičan producer-consumer obrazac koji donosi tri ključne prednosti: vremensko odvajanje (producent i konzument ne moraju raditi istovremeno), apsorpcija opterećenja (Redis služi kao buffer pri naglim skokovima) i otpornost na padove (poruke preživljavaju restart workera).

6 Struktura projekta

```
rs-rently/
|
|- auth-service/
| |- main.py # JWT autentifikacija (FastAPI)
| |- Dockerfile
|
|- booking-service/
| |- main.py # Rezervacije + Redis queue (FastAPI)
| |- Dockerfile
|
|- damage-lambda/
| |- main.py # S3 upload (FastAPI)
| |- lambda-function.py # AWS Lambda handler
| |- Dockerfile
|
|- mail-worker/
| |- main.py # Redis consumer, simulacija e-maila
```

```
| |- Dockerfile
|
|- gps-tracker/
| |- main.py # gRPC server
| |- gps.proto # Protocol Buffers definicija
| |- Dockerfile
|
|- frontend/
| |- src/
| | |- components/
| | | |- Login.vue # Prijava korisnika
| | | |- Dashboard.vue # Pregled sustava
| | | |- Bookings.vue # Kreiranje rezervacija
| | | |- DamageUpload.vue # Upload slika steta
| | | |- CrashControls.vue # Testiranje padova
| | |- api.js # Axios instance + interceptori
| | |- main.js # Vue app, router, Sentry init
| | |- App.vue # Root komponenta s navigacijom
| |- nginx.conf # SPA routing, gzip, cache
| |- vite.config.js
| |- Dockerfile # Multi-stage build (Node + Nginx)
| |- package.json
|
|- docker-compose.yaml # Orkestracija svih servisa
|- start.sh
|- README.md
```

Listing 2: Direktorijska struktura projekta

Svaki backend servis sadrži vlastiti `main.py` s kompletnom logikom i `Dockerfile` za izgradnju kontejnera. Ovakva organizacija osigurava da se svaki servis može razvijati, testirati i deployati neovisno o ostalima. Frontend koristi multi-stage Docker build: prva faza kompajlira Vue aplikaciju putem Node.js-a, a druga kopira generirane statičke datoteke u Nginx kontejner.

7 Docker i pokretanje

Cijeli sustav orkestriran je jednom `docker-compose.yaml` datotekom koja definira osam servisa (šest aplikacijskih i dva infrastrukturna) povezanih zajedničkom Docker bridge mrežom `rent_network`. Unutar te mreže, servisi se referenciraju po imenu – primjerice, Booking-service pristupa Redisu na hostu `redis`, a Damage-service pristupa LocalStacku na `localstack:4566`. Varijable okruženja poput `SENTRY_DSN` i `SENTRY_ENV` prosljeđuju se iz host sustava putem `${...}` sintakse.

Za izgradnju i pokretanje cijelog sustava dovoljne su dvije naredbe:

```
docker-compose up --build # prva izgradnja
docker-compose up # svako sljedece pokretanje
```

Konfiguracija Sentryja. Backend servisi očekuju varijable `SENTRY_DSN`, `SENTRY_ENV` i `SENTRY_TRACES_SAMPLE_RATE` u root `.env` datoteci. Frontend (Vite) čita `VITE_SENTRY_DSN` iz `frontend/.env` tijekom build faze – zbog toga je nakon promjene DSN-a potrebno pokrenuti `docker-compose up -build` kako bi se frontend ponovno kompajlirao s novom vrijednošću.

Nakon pokretanja, sustav je dostupan na sljedećim adresama:

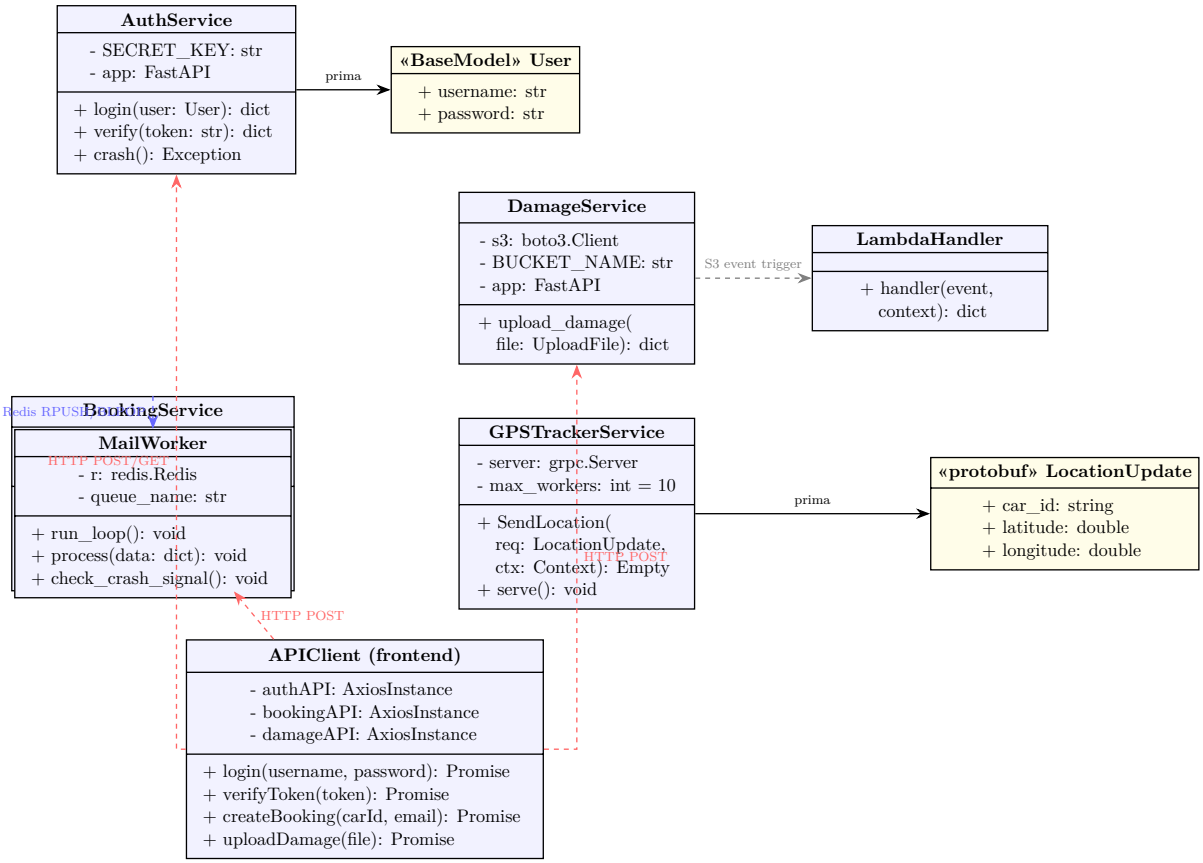
Tablica 2: Mrežna konfiguracija servisa

Servis	**Port**	**Protokol**	**Napomena**
Frontend	3000	HTTP	Nginx, SPA routing
Auth-service	8000	HTTP	Swagger na `/docs`
Booking-service	8001	HTTP	Swagger na `/docs`
Damage-service	8002	HTTP	Swagger na `/docs`
GPS-tracker	50051	gRPC	Protocol Buffers
LocalStack	4566	HTTP	DynamoDB + S3
Redis	6379	TCP	Message broker

Za razvoj frontenda s automatskim osvježavanjem koristi se Vite dev server (`npm run dev` u `frontend/` direktoriju), koji aplikaciju servira na portu 5173.

8 Dijagram klasa

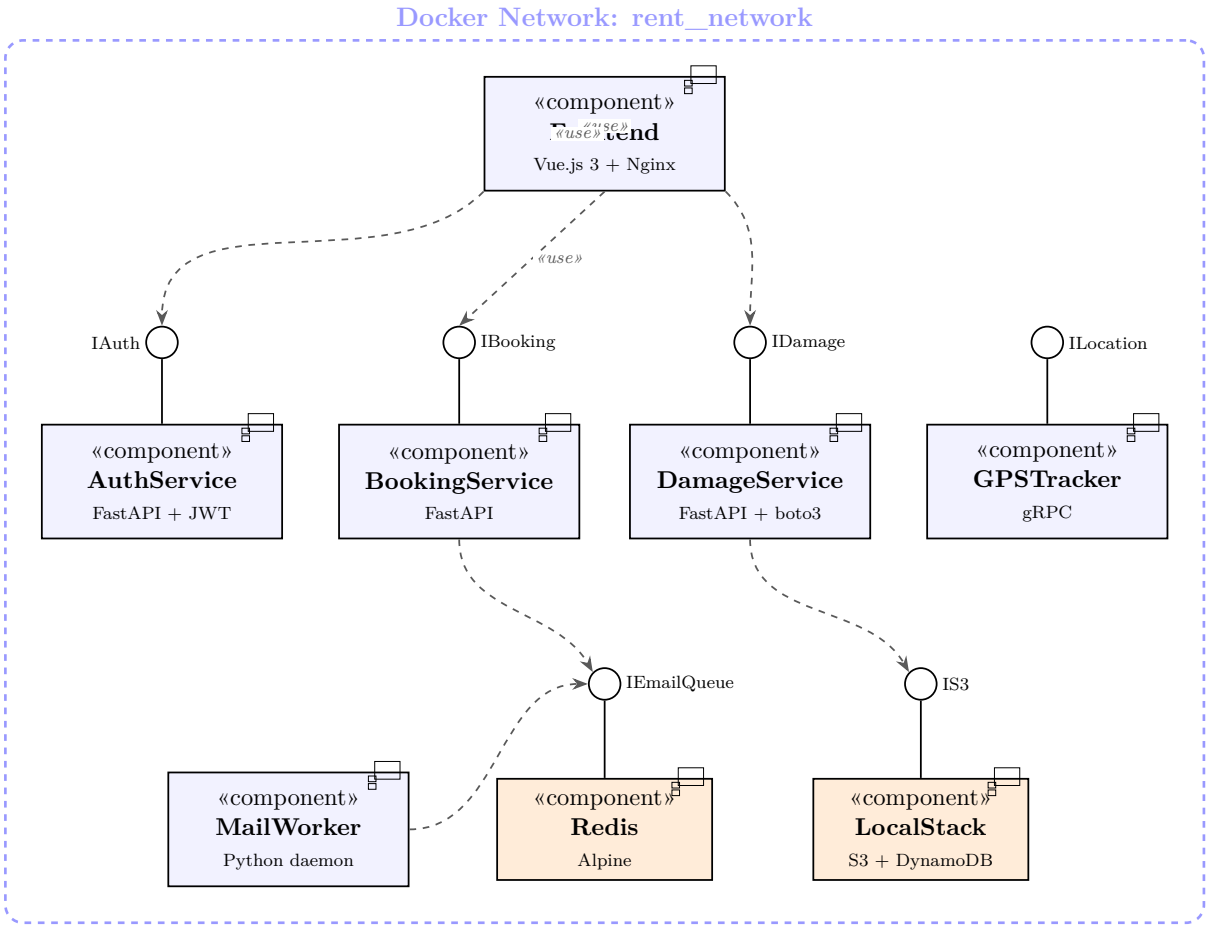
Dijagram na slici 1 prikazuje ključne klase i module sustava te njihove međusobne odnose. Isprekidane strelice označavaju mrežnu komunikaciju (HTTP ili Redis), a pune strelice označavaju izravnu ovisnost.



Slika 1: Dijagram klasa sustava RS-Rently

9 UML dijagram komponenti

Dijagram na slici 2 prikazuje sustav u UML notaciji komponenti. Svaka komponenta označena je stereotipom «component» i UML ikonom komponente u gornjem desnom kutu. Pružena sučelja crtana su kao *lollipop* kružići, a ovisnost potrošača prema pruženom sučelju označena je isprekidanom strelicom sa stereotipom «use». Cijela konfiguracija izvršava se unutar Docker mreže `rent_network`.

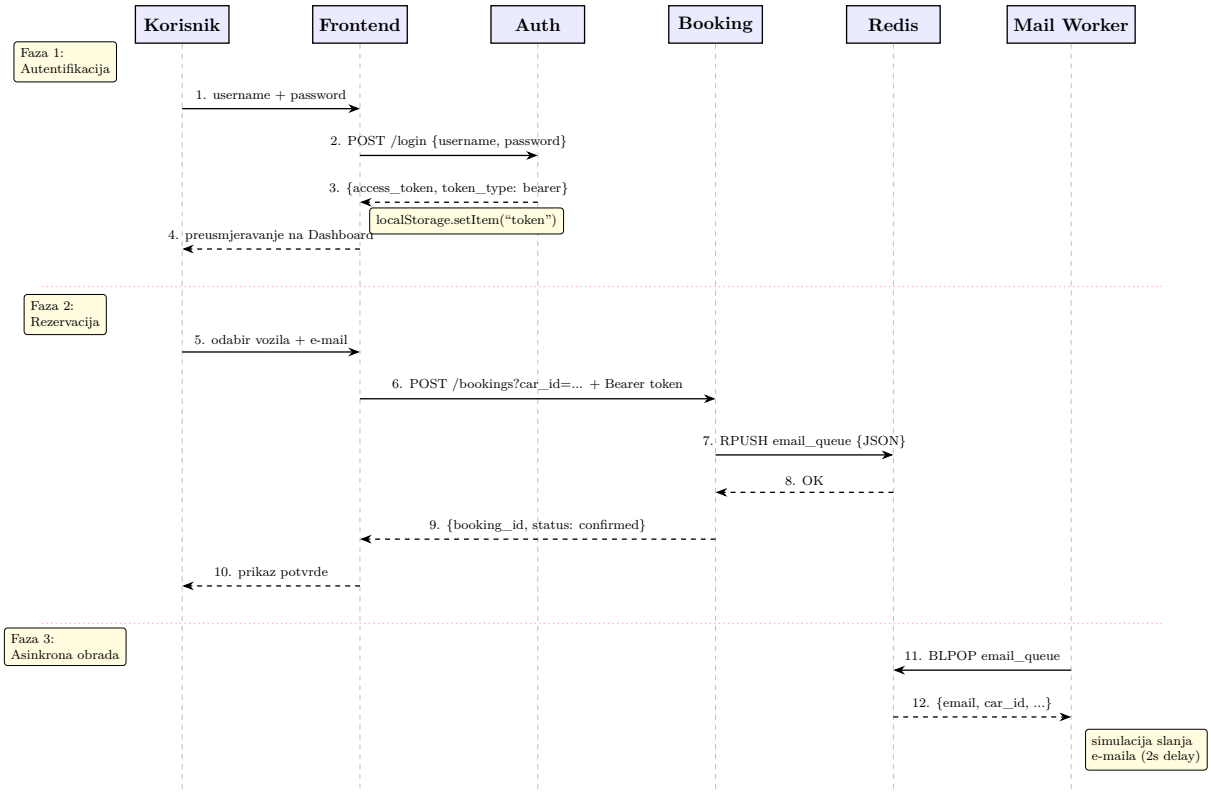


Slika 2: UML dijagram komponenti sustava RS-Rently

Pružena sučelja semantički odgovaraju javnim API-jima pojedinih komponenti: `IAuth`, `IBooking` i `IDamage` su REST sučelja izložena FastAPI-jem, `ILocation` je gRPC sučelje definirano `gps.proto` datotekom, `IEmailQueue` je Redis list API koji koriste producent (BookingService) i konzument (MailWorker), a `IS3` je AWS S3 sučelje koje LocalStack simulira lokalno. Ovakav prikaz jasno razdvaja *što* komponenta pruža od *kako* je implementirana.

10 Sekvencijski dijagram

Slika 3 prikazuje dva osnovna toka sustava. Prvi tok (koraci 1–4) ilustrira autentifikaciju: korisnik unosi kredencijale, Frontend ih prosljeđuje Auth-servisu koji vraća JWT token, a korisnik biva preusmjeren na Dashboard. Drugi tok (koraci 5–11) ilustrira kreiranje rezervacije s asinkronom obradom: Booking-service prima zahtjev, gura poruku u Redis red i odmah vraća potvrdu, dok Mail-worker neovisno preuzima poruku i šalje e-mail.



Slika 3: Sekvencijski dijagram – autentifikacija, rezervacija i asinkrona obrada

11 Zaključak

RS-Rently demonstrira kako se složen poslovni sustav može razložiti na skup neovisnih mikroservisa koji zajedno pružaju cjelovitu funkcionalnost. Svaki servis ima jednu odgovornost, vlastiti životni ciklus i vlastite ovisnosti, a međusobno komuniciraju isključivo putem mrežnih sučelja.

Korištenje tri različita komunikacijska protokola – REST za standardne sinkrone operacije, gRPC za učinkovit prijenos strukturiranih podataka i Redis Queue za potpuno asinkronu obradu – pokazuje da odabir protokola ovisi o prirodi podataka i zahtjevima konkretnog scenarija. REST je intuitivan i univerzalan, gRPC donosi performanse i strogo tipizirana sučelja, a message queue osigurava vremensko odvajanje i otpornost na padove.

Kontejnerizacija putem Dockera i orkestracija putem Docker Compose-a omogućuju pokretanje cijelog sustava jednom naredbom, dok simulacija AWS servisa putem LocalStacka eliminira potrebu za oblačnom infrastrukturom u razvojnom okruženju. Integracija Sentryja na svim razinama – od backenda do frontenda – pruža centralizirani uvid u zdravlje sustava i olakšava dijagnostiku problema.

Ovakva arhitektura otvorena je za daljnje proširenje: dodavanje novih servisa, integracija stvarnih AWS resursa, implementacija Service Mesha ili naprednije analitike ne bi zahtijevali promjene u postojećim servisima, već samo dodavanje novih čvorova u mrežu.